MYSQL Quires

COFFEE SHOP SALES PROJECT

KANIKA JAIN

- Create database coffee_shopsales_db ;
- use coffee_shopsales_db ;
- show tables;
- select*from `coffee_ shop_ sales` ;
- RENAME TABLE `coffee_ shop_ sales`TO coffee_shop_sales;
- select*from coffee_shop_sales ;
- SHOW TABLES LIKE '%coffee%';
- Describe coffee_shop_sales ;
- set sql_safe_updates=0;
- update coffee_shop_sales
  set transaction_date=str_to_date(transaction_date,'%d-%m-%Y');
- alter table coffee_shop_sales
  modify column transaction_date Date;
- update coffee_shop_sales
  set transaction_time=str_to_date(transaction_time,'%H:%i:%s');
- alter table coffee_shop_sales
  modify column transaction_time Time;
- alter table coffee_shop_sales
  change column ï»¿transaction_id transaction_id int ;

 # Total Sales Analysis

 # Calculate the total sales for each respective month

- SELECT ROUND(SUM(unit_price * transaction_qty)) as Total_Sales

  FROM coffee_shop_sales

  WHERE MONTH(transaction_date) = 5 ;

 # or

- select CONCAT(ROUND(SUM(unit_price * transaction_qty) / 1000), 'K') AS total_sales

  FROM coffee_shop_sales

  WHERE MONTH(transaction_date) = 5;

 # Determine the month-on -month increase or decrease in sales

- Select MONTH(transaction_date) AS month,
  ROUND(SUM(unit_price * transaction_qty)) AS total_sales,
  (SUM(unit_price * transaction_qty) - LAG(SUM(unit_price * transaction_qty), 1)
  OVER (ORDER BY MONTH(transaction_date))) / LAG(SUM(unit_price * transaction_qty), 1)
  OVER (ORDER BY MONTH(transaction_date)) * 100 AS mom_increase_percentage
  FROM coffee_shop_sales
  WHERE MONTH(transaction_date) IN (4, 5) -- for months of April and May
  GROUP BY MONTH(transaction_date)
  ORDER BY MONTH(transaction_date);

 # Calculate the difference in sales bettween the selected monthand the previous month

- Select MONTH(transaction_date) AS month,
  ROUND(SUM(unit_price * transaction_qty)) AS total_sales,
  (SUM(unit_price * transaction_qty) - LAG(SUM(unit_price * transaction_qty), 1)
  OVER (ORDER BY MONTH(transaction_date))) as Sales_diff
  FROM coffee_shop_sales
  WHERE MONTH(transaction_date) IN (4, 5) -- for months of April and May
  GROUP BY MONTH(transaction_date)
  ORDER BY MONTH(transaction_date);

 # 2. Total Order Analysis

 # Calculate the total no of orders from each respective month

- SELECT count(transaction_id) as Total_orders
  FROM coffee_shop_sales
  WHERE MONTH(transaction_date) = 5 ;

 # Determine the month_to_month increase or decrase in the no of orders

```
Select  MONTH(transaction_date) AS month,
 ROUND(count(transaction_id)) AS total_orders,
(count(transaction_id) - LAG(count(transaction_id), 1)
 OVER (ORDER BY MONTH(transaction_date))) / LAG(count(transaction_id), 1)
OVER (ORDER BY MONTH(transaction_date)) * 100 AS mom_increase_percentage
FROM coffee_shop_sales
WHERE  MONTH(transaction_date) IN (4, 5) -- for months of April and May
GROUP BY MONTH(transaction_date)
ORDER BY MONTH(transaction_date);
```

 # Calculate the difference in number of orders between the selected month and the previous month

```
Select  MONTH(transaction_date) AS month,
 ROUND(count(transaction_id)) AS total_orders,
(count(transaction_id) - LAG(count(transaction_id), 1)
OVER (ORDER BY MONTH(transaction_date))) as Month_wise_total_orders
FROM coffee_shop_sales
WHERE  MONTH(transaction_date) IN (4, 5) -- for months of April and May
GROUP BY MONTH(transaction_date)
ORDER BY MONTH(transaction_date);
```

 # 3. Total quantity sold analysis

Calculate the total quantity sold for each respective month

```
select sum(transaction_qty) as Total_quantity_sold
from coffee_shop_sales
where month(transaction_date)=5;
```

 # .Determine the Month_on_Month increase or decrease in the total quantity sold

```
select MONTH(transaction_date) AS month,
 ROUND(SUM(transaction_qty)) AS total_quantity_sold,
(SUM(transaction_qty) - LAG(SUM(transaction_qty), 1)
OVER (ORDER BY MONTH(transaction_date))) / LAG(SUM(transaction_qty), 1)
OVER (ORDER BY MONTH(transaction_date)) * 100 AS mom_increase_percentage
from coffee_shop_sales
where MONTH(transaction_date) IN (4, 5)   -- for April and May
group by MONTH(transaction_date)
order by MONTH(transaction_date);
```

#. Calculates the difference in the total quantity sold between the selected month & the previous month

```
select MONTH(transaction_date) AS month,
 ROUND(SUM(transaction_qty)) AS total_quantity_sold,
 (SUM(transaction_qty) - LAG(SUM(transaction_qty), 1)
 OVER (ORDER BY MONTH(transaction_date))) as Month_wise_quantity_sold
 from coffee_shop_sales
 where MONTH(transaction_date) IN (4, 5)   -- for April and May
 group by MONTH(transaction_date)
 order by MONTH(transaction_date);
```

#. Implemented tooltips to display detailed matrics (Sales,Orders,Quantity) when hovering over a specific day

```
select sum(unit_price*transaction_qty) as Total_sales,
sum(transaction_qty) as Total_quantity_sold,
count(transaction_id) as Total_orders
from coffee_shop_sales
where transaction_date = '2023-05-18';
```

 # If we want to get exact query rounded off values below query

```
select concat(round(sum(unit_price*transaction_qty) /1000),'K') as Total_sales,
sum(transaction_qty) as Total_quantity_sold,
count(transaction_id) as Total_orders
from coffee_shop_sales
where transaction_date = '2023-05-18';
```

Sales analysis by weekdays & weekends

 # Segment sales data into weekdays & weekends to analyse performance variations

- ```
  select case when dayofweek(transaction_date)in(1,7) then 'Weekends'
  else 'Weekdays'
  end as data_type,
  round(sum(unit_price*transaction_qty),2) as Total_sales
  from coffee_shop_sales
  where month(transaction_date)=5
  group by case when dayofweek(transaction_date)in(1,7) then 'Weekends'
  else 'Weekdays'
  end;
  ```

 # Using Roundoff

- ```
  select case when dayofweek(transaction_date)in(1,7) then 'Weekends'
  else 'Weekdays'
  end as data_type,
  concat(round(sum(unit_price*transaction_qty)/1000),'K') as Total_sales
  from coffee_shop_sales
  where month(transaction_date)=5
  group by case when dayofweek(transaction_date)in(1,7) then 'Weekends'
  else 'Weekdays'
  end;
  ```

 # Sales Analysis by store location

- select store_location
  (unit_price * transaction_qty) as Total_Sales
  FROM coffee_shop_sales
  WHERE MONTH(transaction_date) =5
  GROUP BY store_location
  ORDER BY SUM(unit_price * transaction_qty) DESC ;

 # Using roundoff and concat

- select store_location,
  concat(round(SUM(unit_price * transaction_qty) /1000),'K')as Total_Sales
  FROM coffee_shop_sales
  WHERE MONTH(transaction_date) =5
  GROUP BY store_location
  ORDER BY SUM(unit_price * transaction_qty) DESC ;

 # Daily sales analysis with Average line

 # Calculating Average sales day wise

- SELECT concat(round(AVG(total_sales)/1000),'K') AS average_sales
  FROM (
  SELECT SUM(unit_price * transaction_qty) AS total_sales
  FROM coffee_shop_sales
  WHERE MONTH(transaction_date) = 5 -- Filter for May
  GROUP BY transaction_date) AS internal_query;

 # Day wise salary

- select day(transaction_date) as day_of_month,
  concat(round(sum(unit_price*transaction_qty)/1000,1),'K') as total_sales
  from coffee_shop_sales
  where month(transaction_date)=5
  group by day(transaction_date)
  order by day(transaction_date);

 # Comparing daily sales with Average sales

- ```
  select day_of_month,
  case when total_sales > avg_sales then 'Above_Average'
  when total_sales< avg_sales then 'Below_Average'
  else 'Equal_To_Average'
  end as Sales_Status,total_sales
  from (select day(transaction_date) as day_of_month,
  concat(round(sum(unit_price*transaction_qty)/1000),'K') as total_sales,
  avg(sum(unit_price*transaction_qty)) over() as Avg_Sales
  from coffee_shop_sales
  where month(transaction_date)=5
  group by DAY(transaction_date) ) AS sales_data
  order by day_of_month ;
  ```

 # Sales analysis by Product Catagory

- ```
  select product_category,
  concat(round(sum(unit_price*transaction_qty)/1000),'K') as Total_sales
  from coffee_shop_sales
  where month(transaction_date)=5
  group by product_category
  order by sum(unit_price * transaction_qty)desc;
  ```

 # Sales product(Top 10)

- ```
  select product_type,
  concat(round(sum(unit_price*transaction_qty)/1000),'K') as Total_sales
  from coffee_shop_sales
  where month(transaction_date)=5
  group by product_type
  order by sum(unit_price * transaction_qty)desc
  limit 10;
  ```

 # sales analysis by top 10 (product type and product category

- ```
  select product_type,
  concat(round(sum(unit_price*transaction_qty)/1000),'K') as Total_sales
  from coffee_shop_sales
  where month(transaction_date)=5 and product_category='coffee'
  group by product_type
  order by sum(unit_price * transaction_qty)desc
  limit 10;
  ```

 # Sales analysis by days & hours .

 # Sales by day -Hour

- ```
  select concat(round(sum(unit_price*transaction_qty)/1000),'K') as Total_sales,
  sum(transaction_qty) as total_quantity,
  count(*) as total_orders
  from coffee_shop_sales
  where dayofweek(transaction_date)=3
  and Hour(transaction_time)=8
  and month(transaction_date)=5 ;
  ```

 # Get sales for all hours for month of May

- ```
  select hour(transaction_time) as Hour_of_day,
  concat(round(sum(unit_price*transaction_qty)/1000),'K') as Total_sales
  from coffee_shop_sales
  where month(transaction_date)=5
  group by hour(transaction_time)
  order by hour(transaction_time);
  ```

 # Get sales from Monday to sunday for month of May

- SELECT

```
    CASE
        WHEN DAYOFWEEK(transaction_date) = 2 THEN 'Monday'
        WHEN DAYOFWEEK(transaction_date) = 3 THEN 'Tuesday'
        WHEN DAYOFWEEK(transaction_date) = 4 THEN 'Wednesday'
        WHEN DAYOFWEEK(transaction_date) = 5 THEN 'Thursday'
        WHEN DAYOFWEEK(transaction_date) = 6 THEN 'Friday'
        WHEN DAYOFWEEK(transaction_date) = 7 THEN 'Saturday'
     ELSE 'Sunday'
    END AS Day_of_Week,
  concat(ROUND(SUM(unit_price * transaction_qty)/1000),'K') AS Total_Sales
 FROM coffee_shop_sales
 WHERE MONTH(transaction_date) = 5 -- Filter for May (month number 5)
GROUP BY
CASE
        WHEN DAYOFWEEK(transaction_date) = 2 THEN 'Monday'
        WHEN DAYOFWEEK(transaction_date) = 3 THEN 'Tuesday'
        WHEN DAYOFWEEK(transaction_date) = 4 THEN 'Wednesday'
        WHEN DAYOFWEEK(transaction_date) = 5 THEN 'Thursday'
        WHEN DAYOFWEEK(transaction_date) = 6 THEN 'Friday'
       WHEN DAYOFWEEK(transaction_date) = 7 THEN 'Saturday'
 ELSE 'Sunday'
END;
```